

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندوی	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for: Community, NGO or village work

Contents

Part I: Orientation 2

- 2. The work, not the worker 3
- 3. Start here: find your way in 4

Part II: One architecture 6

- 10. Three families, set apart: the AyeAI Triad, GramSheel and Project VIKRAM, 7
Humanesque and its projections

Part V: Working with the estate 12

- 28. Questions that come up, answered technically 13

Appendices 17

- E. For practitioners: teachers, policy makers, communities, makers and engineers 18

Part I: Orientation

2. The work, not the worker

In plain words. What matters is whether the work holds up, whether the code runs and the reasoning is sound, not who made it. The aim is that people build technology in their own languages.

We know Pāṇini almost entirely through his grammar. Tradition gives a birthplace, Śalātura in Gandhāra, and very little else that can be checked. Close to four thousand sūtras survive; the person does not. The grammar still works, and Chapter 20 runs part of it. That is the standard this guide keeps. Work has to stand without its author, because titles, affiliations and biographies do not compile, and code, derivations and measurements do.

Two consequences follow for how the estate is built.

The first is enablement. Linguistic and cultural equity is not achieved by making AI available everywhere. Systems have to work within the languages, scripts, cultural contexts and knowledge traditions of the people they serve, so that those people become producers of technology in their own languages rather than remaining users of someone else's.

The second is openness. Every repository in Chapter 12 can be read, run, forked and criticised. Nothing here needs anyone's permission to use; each repository states its licence.

The programme stands on two pillars. The linguistic pillar standardises at the level of the dialect, with the idiolect as a later overlay, and grows through linguists working across the world. The technological pillar, the older of the two, is now the enabler: the transliteration engines, compilers, parsers and substrates that make a language usable for building. This guide deals mostly with the second pillar, because that is where a newcomer can start contributing today.

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part II: One architecture

10. Three families, set apart: the AyeAI Triad, GramSheel and Project VIKRAM, Humanesque and its projections

In plain words. Three groups explained one at a time: the AyeAI Triad of thinking, body and communication; GramSheel and Project VIKRAM for villages; and Humanesque with the names it takes in each culture.

Three families of work are easy to blur into each other or into the rest of the estate. This chapter sets each apart, with what its own record says.

The AyeAI Triad

The Triad has three members, AyeAI®, AyeAM® and AyeCNSe®, with AyeAI at the apex and AyeCNSe and AyeAM at the base. It is a triad, not a pipeline and not a loop: a loop running from the environment through AyeCNSe, AyeAI and AyeAM back to the environment was proposed elsewhere and rejected. Its formal closure gives each member a tuple:

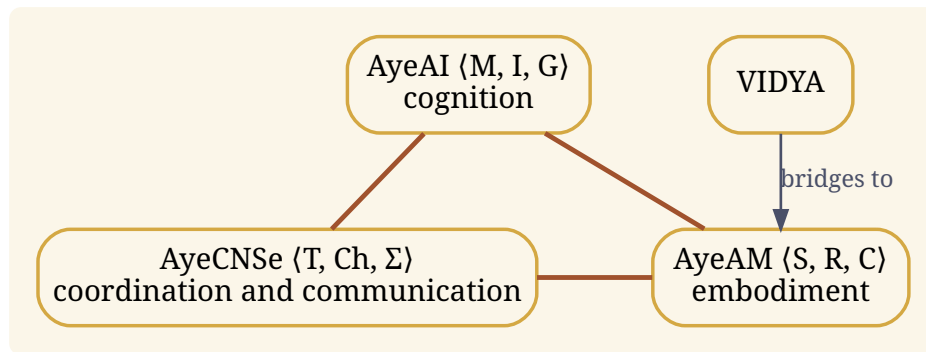


Figure 8. The AyeAI Triad: a triad with AyeAI at the apex, not a loop and not a pipeline.

$$\mathbf{AyeAM} = \langle S, R, C \rangle, \quad \mathbf{AyeAI} = \langle M, I, G \rangle, \quad \mathbf{AyeCNSe} = \langle T, Ch, \Sigma \rangle.$$

AyeAI is cognition, the apex: “the inclusive AI”, in the words of its site, and the organisation [ayelai](#) holds its platforms, among them an open public stack for synthetic intelligence (opssi), virtual deployment infrastructure for scientific and cognitive computing (ayevdi), Staggered Upload™ (upload), the AyeAI Singularity Public License (spl) and the NI2A2 work.

AyeAM is embodiment: the body through which intelligence acts, across ground, air, water, space and habitat. VIDYA bridges to AyeAM without being a member of the Triad.

AyeCNSe is coordination and communication across media: the form studio at [forms.ayecnse.site](#) and the Human Context Model paper at [hcm.ayecnse.site](#) are its published faces.

Two repositories mark themselves as PEDLER and AyeAI Triad work: PRATIK’s kernel and Zamin’s ternary silicon.

GramSheel and Project VIKRAM

The GramSheel Foundation is the social architecture of the estate and the older of its lines: † it began as the Nature Welfare Society in 1993. Its principles, HEJLP, are health, education, justice, livelihood and peace. Its Village Knowledge Center (Universal Device) was a Stockholm Challenge finalist in 2010, and TWISHA runs under the same foundation.

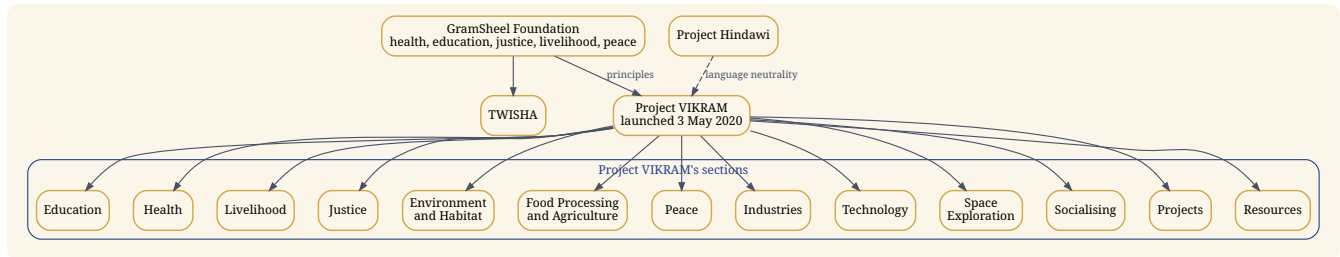


Figure 9. GramSheel and Project VIKRAM, with VIKRAM's sections as its own pages name them.

Project VIKRAM, Virtualized Infrastructure for Knowledge-driven Rural Ascension Management, is † the technical implementation of GramSheel's principles. It was launched on 3 May 2020 as the Project Vikram Journal, “an inclusive rural digitalization platform inspired by the principles of GramSheel”, under the motto गाँव बढ़ेंगे तो सब बढ़ेंगे, when the villages grow, everyone grows. Its own pages (pvjournal.github.io) name its components:

Component	What its page covers
Education	learning and schooling
Health	healthcare, open to submissions as issues, with tools for an accessible virtual hospital
Livelihood	the ISIC domains of economic activity
Justice	social justice
Environment and Habitat	the living environment
Food Processing and Agriculture	agriculture and food
Peace	harmony through social dialogue
Industries, Technology and Space Exploration	the technical frontiers it reaches towards
Socialising, Projects and Resources	the community and its shared material

With Project Hindawi, its page says, VIKRAM works towards language neutrality across technical fields. In the three Acts, GramSheel and Project VIKRAM stand on the divider between Acts II and III.

Gram Swaraj and the questions behind GramSheel and VIKRAM

Gandhi's *Hind Swaraj* (1909) argued that self-rule is more than a transfer of authority, and his *Constructive Programme: Its Meaning and Place* (1941) placed the rebuilding of daily life inside the work of independence: its eighteen items include village industries, village sanitation, basic and adult education, the provincial languages and a national language, economic equality, the kisans, labour, the adivasis and women. The estate does not present itself as a restatement of these ideas. The point of contact is a question that becomes unavoidable once computation is infrastructure: can a community govern itself if the knowledge and the machines its decisions depend on are controlled elsewhere?

On Gandhi Jayanti, 2 October 2026, the author put the questions behind GramSheel, Project VIKRAM and Humanesque in his own words. Among them:

- Can rural communities become producers of knowledge and technology, not merely consumers of systems designed elsewhere?
- Can technology distribute capability without concentrating power?
- Can artificial intelligence strengthen individual and community sovereignty rather than replace it?
- Can knowledge remain plural, multilingual and culturally grounded while becoming computationally executable?
- Can verification, provenance and human agency become constitutional properties of intelligent systems?
- Can GramSheel build the local knowledge infrastructure, can Project VIKRAM turn that infrastructure into practical capability, and can Humanesque provide a larger architecture in which intelligence, identity, agency, provenance and equity remain connected?

His answer to the old division of the world is a direction rather than a side: to march together, not as a Global South or a Global North, but towards a Global Centre, where capability is distributed and humanity remains sovereign.

Humanesque and its projections, with their names

Humanesque is the core, with its site at humanesque.site and its organisation at hmnsq: the shared executable typed hypergraph, the ontology federation, the common execution and evidence spine, and the components every projection reuses. Its constitutional ground is PoIE, from which sovereignty recurses to every level. Its core stem repositories all belong to Humanesque; each projection gives them its own culturally sensitive names and sites.

Kaivalyik, the Indic projection, including the Advaita framing (kaivalyik.org, organisation [kaivalyik](https://kaivalyik.org)): consciousness, intelligence and transcendence; it carries Synthemetic Pragmatic Realism, Kaivalyik AGI and Jyotish. **Zistgah**, زیستگاه, the Persian and Islamicate projection (zistgah.org, organisation [zistgah](https://zistgah.org)): life, planet and ecology, a habitat to live and work in; it carries the elements, the dome and the habitats on Earth (Asli Zistgah), the Moon (Zistgah-e-Mahtab) and Mars (Zistgah-e-Bahram, † conceived earlier as Mangal Kamna). **Cosmopolis**, the Greek, Western and classical projection (cosmopolis.org, organisation c-polis): societies, civilisations, space and deep time.

The stem names below are the projection-neutral names of the components. Today the working repositories live in the zistgah organisation, so most of them carry their Zistgah names; two stems already show a second projection, HPS in Hindawi and CHAKRA in Jyotish.

Stem, in Humanesque	Zistgah	Kaivalyik
The programming system	Urdu edition (urdu-ilm)	Hindawi
The observatory and its calendars	Chakra, the turning sky	Jyotish
The ground: balanced-ternary silicon	Zamin, زمین	
The water: the factory process	AAB, آب	
The air: the environmental replica	Fiza, فضا	
The shared virtual dome	the dome	
The Cognitive Workbench	Mez, میز	
The press	Matba, مطبع	
The cutting room, the visual cyler	Khwab, خواب	
The listening room, the audio cyler	Awaz, آواز	
The immersive cyler	Tilasm, طلسم	
The embodied cyler	Pench	
The record	Yadein, یادیں	
The book template	Kitab, کتاب	
The domain spine	Dhancha, ڈھانچہ	
Communication engineering	Ertabat, ارتباط	
Device control	Zasab, اعصاب	
The door for any AI	Darwaza, دروازہ	
Scan and store	Tasvir, تصویر	
The plates	Lawh, لوح	
The parser federation	Tajziya, تجزیہ	
The guide	Rahnuma, رہنما	
The Geometry of Becoming	Tabdili, تبدیلی	
The Geometry of Living	Zindagi, زندگی	
The Geometry of Persisting	Qaem, قائم	
Topological Bildung	Tarbiyat, تربیت	

Some stems keep one name in every projection: Humanesque, ILM, Romenagri, PANINI with PANINIq, PANINIb and PANINIphy, PEDLER, QEDLER, PRATIK, CEM, UKOP, FAKIR, GENIE, TransEg, PoIE, VGC, COPA, and the provenance systems Tok DOI, spiguard, Candor and Misty DOI. Kaivalyik's and Cosmopolis's own names for the rest are given with their organisations, as each projection takes its repositories.

Part V: Working with the estate

28. Questions that come up, answered technically

In plain words. Short technical answers to questions people often ask, such as whether AI needs English, and who controls it.

These questions are asked whenever the work is discussed. Each answer points to the chapter that carries the evidence.

If it uses GCC, or Qiskit, does it not still depend on foreign technology?

A compiler is front ends and back ends joined by an intermediate representation (Chapter 21). By the time a program reaches GCC's GIMPLE, a loop written with `for` and the same loop written with `क्रम` are the same basic blocks; the language of the keywords is settled and discarded at the front end. What does reach the back end is the program's names, and Hindawi hands those over in Romenagri, so that they come out of the binary intact and map back to the script (Chapter 21). Reusing a mature back end is how Rust, Swift and Julia were all built on LLVM, and nobody calls them dependent on C++. PANINIq hands Qiskit a Hamiltonian, which is a matrix of complex numbers. The dependency that does matter is on source code and on people who can rebuild it, which is the next answer.

What would a sovereign stack actually require?

Source code for every layer, builds that reproduce bit for bit, independent verification of the compilers by diverse double-compiling, and enough trained people to rebuild, verify and modify each layer (Chapter 21). None of this depends on the alphabet of the keywords, and all of it depends on skills. That is why the next steps here are courses and laboratories, not a logo.

Is this not white-labelling borrowed technology?

White-labelling sells someone else's product under a new brand, often closed. Everything here is the opposite: public source under the GPL, dated DOIs, and forks and pull requests invited (Chapter 25). What is new is stated precisely: the language front ends, the construct model with its decorators, the reversible transliteration, the transducers and the substrates. What is reused is reused openly and named, as every serious compiler project reuses back ends, libraries and standards.

Are Unicode and InScript not enough?

Unicode encodes characters and InScript is a keyboard layout. Together they let you type and store Indic text (Chapter 18). Neither translates keywords, libraries, compiler messages or documentation; neither tells a program which language a Devanagari string is in; and neither makes a name written in the script survive an assembler, a linker script, an ELF symbol table, DWARF, a debugger and a JTAG probe and come back readable. The first two gaps are the language front end's; the third is the script layer's, Romenagri's (Chapters 19 and 21).

Is a script not the same as a language?

No. Punjabi is written in Gurmukhi and in Shahmukhi; Devanagari writes Hindi, Marathi, Nepali and Sanskrit, and pronounces the same letters differently in each (Chapter 19). A system that ties language to script cannot grow past its first language, which is why ILM keeps script, language and standard on separate axes.

Should there not be a conference first?

A conference is where results are presented and checked against each other, and results begin with running code. The repositories are open now, so the most useful contribution anyone can make today is an issue or a pull request against something that runs. A conference with running code behind it has something to test.

Is 2047 the horizon, or 2030?

The number of people who can build decides the date. Courses at GATE level and reference labs are how that number grows, which is why they come next (Chapters 16 and 26). The estate's own schedule is in Chapter 31.

Is quantum hardware not out of reach?

Much less than it looks. A laptop simulates about thirty qubits exactly (Chapter 23), and PANINIq runs its quantum walk and QAOA baseline on one. Cloud services give access to real quantum processors, some with free tiers. Oscillator Ising machines can be built from ordinary electronics, and chips can be designed and taped out on open process design kits (Chapter 22). What is out of reach is building a dilution refrigerator from scratch, and that is not where the work starts.

What has the Indian knowledge tradition contributed that is technical?

Specific, checkable things. Pāṇini's grammar is a formal generative system with character classes, ordered rewrite rules and meta-rules for conflict (Chapter 20). Piṅgala's *Chandaḥśāstra* enumerates the patterns of long and short syllables in metres with rules equivalent to counting in binary, and its *meru-prastāra* is the triangle of binomial coefficients later named after Pascal. Brahmagupta stated rules for arithmetic with zero in 628 CE. Āryabhaṭa's *kuṭṭaka* solves linear equations in integers, the problem the extended Euclidean algorithm solves. Mādhava of Saṅgamagrāma, in the fourteenth century, gave infinite series for the arctangent, sine and cosine, including

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots,$$

about two centuries before they appeared in Europe. Where a tradition is said to be not yet understood, the useful response is the one Chapter 20 takes: encode it, run it, and let the output speak.

Does AI depend on ASCII, and would it stop working without it?

There is no separate “ASCII foundation” to cut off. ASCII is the first 128 code points of Unicode and the one-byte range of UTF-8 (Chapter 18). A current language model reads text as tokens built over UTF-8 bytes or Unicode characters, and it processes Devanagari, Urdu or Chinese through the same machinery as English.

What does exist is an asymmetry, in three places. The training data are mostly English. The tokenisers, trained mostly on English, cut Indic text into more tokens for the same content: every Devanagari letter is already three bytes in UTF-8, so the same sentence costs more to process and fills the context window sooner. And the toolchains beneath the models keep their keywords, libraries and symbol alphabets in ASCII (Chapters 18 and 21). None of the three is a law to surrender to. Each has a technical remedy: more native text, tokenisers with script-aware merges and, at the toolchain, Romenagri, which uses ASCII-7 as a carrier while the script’s identity stays reversible.

Romenagri’s own measurements are recorded in [project-ilm/romenagri](#) and should be quoted as they stand. Written in Romenagri, the test corpus takes 1.76 times fewer bytes, and its alphabet shrinks from 51 symbols to 24, with nothing lost. In the recorded tokeniser sweep, however, Romenagri did not reduce the number of tokens at matched numbers of merges: the ratio ran from 0.62 to 1.0. The three- to five-fold token gain predicted in the author’s tokenisation paper therefore stands as a falsifiable prediction still to be shown, and his runbook asks for exactly this result to be reported, not hidden.

Who owns the leading AI engines, and does ownership decide which languages and nations are sovereign?

Ownership of the leading models changes by the quarter and is not an engineering property; summaries of it, including AI-generated ones, need checking against primary sources before they are repeated. What owners decide is what their own tools do. They do not decide what a community can build. In the technical sense of Chapter 21, sovereignty belongs to whoever can rebuild, verify, modify and run the stack from source. The estate is built to that standard and to be independent of any one model: the cyclers run with any AI, Romenagri, ILM and PANINI are open and run on a laptop, and open-weight models can be run locally (Chapter 24). A community that holds its scripts’ path through the toolchain, its tokenisers and its corpora does not depend on who owns the most popular engine.

What is the state of the art for Indic and Chinese text in AI?

The field moves monthly, so this answer gives the mechanisms rather than model names and scores, which should be taken from published model cards and papers. Chinese and the Indic scripts pose different problems. A Chinese character is a single code point, three bytes in UTF-8, and usually carries a morpheme, so the question is how large a vocabulary of characters and words a tokeniser can afford. An Indic akshara is a cluster of several code points (Chapter 18), so the question is whether the tokeniser keeps clusters and morphemes whole. The approaches in use are larger vocabularies with merges learned from native text, pre-tokenisation that respects grapheme clusters, byte-level and character-level models, more native training data, and evaluation written in the languages themselves rather than translated from English. Reversible transliteration such as Romenagri is a further route, with the measured gains and limits given above.

What is a software factory, and what is the estate's?

The idea is older than most of the tools that now revive it. Douglas McIlroy proposed mass-produced software components at the NATO software engineering conference of 1968, and Hitachi organised software production as a factory in Japan in 1969; Cusumano's *Japan's Software Factories* (1991) is the history. The core is a universal set of software components that can be synthesised architecturally into a working product, each component keeping its provenance, which in present terms means its characterisation and its contracts. University courses and industry writing have taken the idea up again, for example themodernsoftware.dev and [factory.com's account](https://factory.com).

The estate builds its own version. **FAKIR** is the component registry: its categories follow the international classifications of the UN and the ILO, ISIC for economic activity, ISCO for occupations and ISCED for fields of education and learning, so that every lawful human economic activity has a place. **AAB** is the factory process: a paint program for systems, from painted intent to verified, sealed artifact. In the author's words, AAB and FAKIR together constitute the working definition of AGI used here. What most current work leaves out is the provenance of the components; here the registry records identity, provenance records ancestry and transformation, and evidence establishes what may be claimed. A release is quality-gated; beta versions are open on GitHub. The landing page at zistgah.org/fakir shows the component categories as a cloud near the ceiling of the dome, some 11,000 of them by his count, with a quadrotor to fly through them; passing through a component lets you start building it and send the work back as a pull request. The process has its own page at zistgah.org/aab.

How should an AI's confident answer be judged?

By what it has run, not by how certain it sounds. **COPA**, the Cost of Perceived Authority (doi:10.5281/zenodo.21782217), is the estate's protocol for the pattern: a black-box behavioural method for inferring the effective operational objective of a deployed AI system. Its metric, the Authority Projection Index, is the number of explanatory tokens before the first empirical action divided by the total number of tokens. COPA states its hypothesis together with a fair null and does not treat a single reading as proof of intent. What it establishes regardless of cause is the cost of assertion before verification: automation bias, overtrust, anchoring, delayed checking and reduced scrutiny. The target is calibrated trust, and the working rule follows from it: ask the AI to run the thing before you accept its explanation.

Can I point any AI at Romenagri and have it check the claims?

Yes, provided the AI can execute code: Claude with code execution, Claude Code, Codex CLI, Gemini CLI or Aider on your own machine (Chapter 24). Give it this:

Clone <https://github.com/hindawiai/chintamani>. Read [Romenagri/](#) and [Hindawi/hindrv/hincc](#). Build Romenagri with `make`, then the guru shaili and the driver, installing into a folder inside the clone. Run a few Devanagari words through `uni2acii | acii2cf` and back through `rmn2acii | acii2uni`, and compare. Compile [Hindawi/samples/HindiC.uhin](#) with `hincc` and run it. Then build it with `-g` and show the names in `nm` and in GDB. Show every command and its output, and report nothing you did not run.

Chapter 15 gives the same steps as commands, and Chapters 19 and 21 show what they print.

Appendices

E. For practitioners: teachers, policy makers, communities, makers and engineers

In plain words. A guide for teachers, policy makers, community workers, artists and engineers: where to start and how to take part.

Teachers and lab builders. Start from the reference lab, the GATE chapter and the component cards, which double as laboratory sheets: each says what to install, what to type and what a student should see (Chapters 16, 26 and 15). The reader views let each student follow a path suited to their age and background, and remember what they have read.

Policy makers and administrators. The questions that matter are in Chapter 10 and Appendix C: sovereignty in its testable technical sense, the ability to rebuild, verify, modify and run (Chapter 21); provenance that makes claims checkable; and infrastructure that leaves capability with communities. The executable contracts of the repositories show what verifiable governance of software looks like in practice.

Community workers and health practitioners. Project VIKRAM's sections cover education, health, livelihood, justice, environment and habitat, food and agriculture, and peace; its health section accepts submissions as issues. The Urdu edition and the Brahmi hub show work in people's own scripts; CHAKRA and Jyotish run offline.

Artists, writers and makers. The cyclers produce print, film, sound, immersive work, embodied work and the record; matba turns posters into a sealed book; GENIE writes and paints; PANINIphy turns a physical idea into parts. Every cycle is recorded as method, so a piece of work can be repeated, shared and credited.

Engineers in industry. Adopt one component at a time: read its card, clone it, run it, read its `CONTRACT.md` and its gate, check its DOI and provenance, then integrate it through its interface. † The notes give the full adoption ladder: the canonical artifact, a one-page explanation, the nearest industry analogue and the exact difference, a working demonstration, the repository, the DOI and timestamp, the interface, a case study, and the commercial route.

Contributors. Fork, open an issue, send a pull request. Every repository's gate runs on your change; every claim must point to its evidence; the AI you use is welcome, and its report that something passed is not the evidence that it did.

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.